

Sistema de Controle de Acesso Biométrico com Processamento Local e Defesa contra Ataques de Apresentação

Fernando de Melo Colli – 231026349
Faculdade de Ciências e Tecnologias em Engenharia (FCTE)
Universidade de Brasília

Resumo—Este trabalho apresenta o desenvolvimento de um sistema de controle de acesso residencial baseado em visão computacional e hardware embarcado, executado integralmente sobre uma Raspberry Pi 3 Model B com foco em autonomia e privacidade (*Edge Computing*). Neste estágio (PC3), o protótipo evoluiu de módulos isolados para um sistema integrado de tempo real, estruturado em dois processos independentes que se comunicam por um *pipe* nomeado (FIFO): um núcleo de controle de baixo nível escrito em C++ — responsável pelo acionamento de uma trava solenoide de 12 V e pela sinalização de estados em LED RGB — e uma *pipeline* de visão em Python/OpenCV responsável pela detecção, prova de vida (*liveness*) e reconhecimento de identidade. O núcleo C++ emprega múltiplas *threads*, exclusão mútua (*mutex*) e a chamada de sistema `poll()` para sincronização eficiente, eliminando *busy-wait*. A prova de vida combina quatro métricas (textura LBP, variância Laplaciana, gradientes de Sobel e análise de frequência via FFT) integradas por uma janela temporal deslizante. Os resultados confirmam alta robustez do módulo *anti-spoofing* e estabilidade da comunicação entre processos, além de discutir os desafios de calibração do classificador biométrico sob variações de iluminação e uma anomalia elétrica observada após o encapsulamento do protótipo.

I. INTRODUÇÃO

A segurança residencial atual demanda autonomia e privacidade. Diferente de soluções comerciais como Ring [1] ou Nest [2], que dependem de servidores externos, este projeto foca no processamento local (*Edge Computing*), mantendo todo o dado biométrico dentro do dispositivo.

Utilizou-se a biblioteca OpenCV [3] para manipulação de vídeo e o algoritmo de Haar Cascades [4] para detecção facial. Para mitigar variações de iluminação, aplicou-se a técnica CLAHE [5], que melhora o contraste local. A defesa contra fotos e telas (*anti-spoofing*) é feita por análise de textura *Local Binary Patterns* (LBP) [6], enquanto o reconhecimento da identidade é realizado pelo classificador LBPH [7].

O sistema implementa uma portaria eletrônica com reconhecimento facial. Além da autenticação do morador, a arquitetura foi concebida para suportar um mecanismo de **senha temporária para entregadores** (por exemplo, pedidos de iFood), em que um código de uso único e com validade limitada libera o acesso sem expor a credencial biométrica — funcionalidade prevista a partir da sugestão de avaliação do PC1 e detalhada como trabalho futuro na Seção IV.

O foco deste terceiro ponto de controle (PC3) é o **refinamento e a integração do protótipo**: conectar as partes

fundamentais (visão, controle e atuação) em um sistema único e introduzir recursos básicos de sistema operacional — múltiplos processos, *threads*, *pipes*, sinais e exclusão mútua (*mutex*) — empregando as linguagens C e C++ no núcleo de tempo real [8]. As seções seguintes descrevem o hardware refinado (II-A), a arquitetura de software concorrente (II-B) e os resultados experimentais obtidos (III).

II. DESENVOLVIMENTO DO PROJETO

A. Descrição de Hardware

A arquitetura de hardware foi projetada para garantir a integridade da unidade de processamento e a eficiência no acionamento de cargas de potência. O sistema utiliza uma Raspberry Pi 3 Model B como núcleo central, integrada a periféricos de captura e atuação. Em relação ao estágio anterior, o atuador foi consolidado em uma **trava solenoide de 12 V** (o servo motor foi descartado), por oferecer acionamento do tipo tudo-ou-nada mais adequado a uma fechadura elétrica. A Tabela I apresenta a lista de materiais atualizada.

Tabela I
LISTA DE MATERIAIS (BOM) – IMPLEMENTAÇÃO REAL (PC3)

Componente	Quantidade
Raspberry Pi 3 Model B	1
Trava Solenoide 12 V	1
Fonte de Alimentação 12 V	1
Transistor TIP31C (NPN, potência)	1
Transistor BC548 (NPN, driver)	1
Diodo Retificador 1N4007 (<i>flyback</i>)	1
Resistor 330 Ω	4
Resistor 10 k Ω	1
LED RGB de cátodo comum	1
Capacitor Eletrolítico 100 μ F	1
Webcam Logitech Brio 100 (USB)	1

1) *Diagrama de Blocos*: O diagrama de blocos da Fig. 1 apresenta os principais componentes de *hardware* e a relação entre eles. Para diferenciar a natureza de cada conexão, os fluxos de **sinal e dados** são representados por setas finas, enquanto os fluxos de **energia e potência** são representados por setas grossas duplas, conforme a legenda.

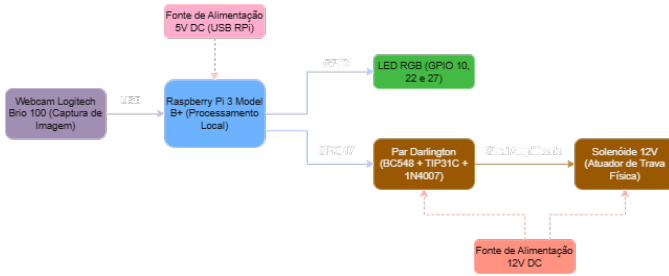


Figura 1. Diagrama de blocos do sistema: componentes de hardware.

2) *Estágio de Potência e Esquemático*: Como a Raspberry Pi opera com lógica de 3,3 V e possui limitação de corrente nas GPIOs, implementou-se um estágio de *driver* em configuração Darlington unindo os transistores BC548 e TIP31C, capaz de chavear a corrente da solenoide. O sinal proveniente da GPIO 17 passa por um resistor de 330 Ω até a base do primeiro transistor; um resistor de *pull-down* de 10 k Ω garante o estado desligado na ausência de sinal. Um diodo *flyback* 1N4007 é colocado em antiparalelo com a bobina para suprimir o pico de tensão reversa no desligamento, e um capacitor de 100 μ F filtra transientes no barramento de 12 V. O esquemático elétrico — apresentado de forma profissional na Fig. 2, substituindo a montagem de *protoboard* do estágio anterior — detalha estas conexões.

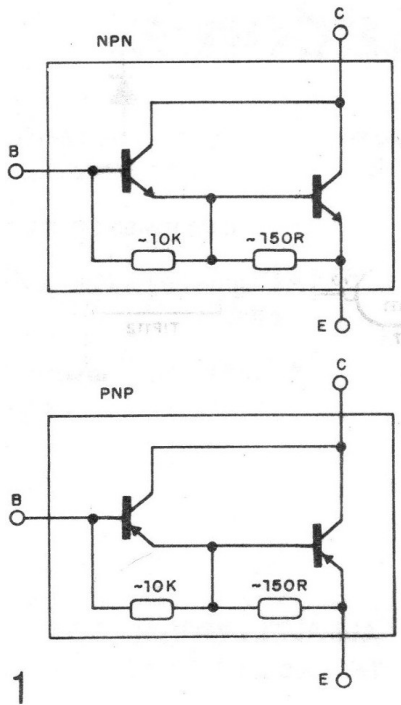


Figura 2. Esquema elétrico do estágio de *driver* de potência Darlington para a solenoide, com diodo de roda-livre 1N4007 e sinalização em LED RGB.

Destaca-se que o GND da fonte de 12V e o GND da Raspberry Pi devem compartilhar um **barramento de terra comum**; a ausência ou o rompimento desse ponto comum é uma falha crítica, discutida na Seção III.

B. Descrição de Software

A principal evolução do PC3 está na arquitetura de software, que deixou de ser um único script monolítico e passou a ser um **sistema concorrente de dois processos** sincronizados em tempo real, explorando recursos básicos do sistema operacional Linux [8]:

- **Processo de Visão (Python 3 / OpenCV)** — captura de vídeo, detecção facial, prova de vida (*liveness*) e reconhecimento de identidade;
- **Núcleo de Controle (C++)** — controle de baixo nível do hardware (solenoide e LED RGB), executando *múltiplas threads* sincronizadas por *mutex*.

1) *Comunicação entre Processos (IPC)*: A sincronização entre os dois processos é feita por um *pipe* nomeado (FIFO) em `/tmp/gate_pipe`. O processo Python escreve um único byte para sinalizar o veredito: 'F' para usuário autorizado (abre a solenoide por 4 s e acende o LED verde) ou 'I' para alerta de intruso/fraude (dispara o pisca-pisca vermelho/azul por 6 s e envia notificação com foto pela API do Telegram). A escrita é *não-bloqueante* (`O_WRONLY | O_NONBLOCK`), como mostra a Listagem 1.

```
def enviar_sinal_pipe(sinal: bytes):
    try:
        fifo = os.open(PIPE_PATH, os.O_WRONLY | os.
            O_NONBLOCK)
        os.write(fifo, sinal) # 'F' = autorizado | 'I' =
            intruso
        os.close(fifo)
    except OSError as e:
        print(f"[PIPE] Falha ao enviar '{sinal}': {e}")
```

Listing 1. Escrita não-bloqueante do veredito no FIFO (Python).

No lado C++, uma *thread* dedicada mantém o descritor de leitura do FIFO aberto de forma persistente e usa a chamada de sistema `poll()` com *timeout* de 100 ms. Isso bloqueia a execução de forma eficiente até a chegada de dados, eliminando *busy-wait* e reduzindo drasticamente o uso de CPU da placa (Listagem 2).

```
fd = open(PIPE_PATH, O_RDONLY | O_NONBLOCK); // leitor
persistente
while (rodando) {
    struct pollfd pfd{fd, POLLIN, 0};
    int ret = poll(&pfd, 1, 100); // bloqueio
    // eficiente (100 ms)
    if (ret <= 0) continue; // timeout:
    // revisa flag "rodando"
    if (pfd.revents & POLLIN) {
        char comando; read(fd, &comando, 1);
        if (comando == 'F') { /* abre solenoide 4 s + LED
            verde */ }
        else if (comando == 'I') { /* alerta de intruso 6 s
            */ }
    }
}
```

Listing 2. Monitoramento eficiente do FIFO com `poll()` (C++).

2) *Concorrência, Mutex e Sinais*: O núcleo C++ executa duas *threads* (`std::thread`): `threadMonitorPipe`, que escuta o FIFO, e `threadControleLED`, que atualiza a sinalização. O estado compartilhado (`portaAberta`, `alertaIntruso`) é protegido por um `std::mutex`, enquanto um segundo `mutex` (`hwMtx`) serializa o acesso ao hardware via `pinCtrl`, evitando condições de corrida na

escrita das GPIOs. Adicionalmente, o programa trata os **sinais** SIGINT e SIGTERM para desligar o hardware de forma limpa (Listagem 3). O Quadro II resume o mapeamento entre os requisitos de sistema operacional do PC3 e sua implementação.

```
void escreverGPIO(std::string pin, int valor) {
    std::lock_guard<std::mutex> lock(hwMtx); // MUTEX de hardware
    // ... aplica o nível via pinctrl ...
}
void tratarCtrlC(int sinal) { rodando = false; } // handler de sinal
signal(SIGINT, tratarCtrlC);
signal(SIGTERM, tratarCtrlC);
```

Listing 3. Exclusão mútua no acesso ao hardware e tratamento de sinais (C++).

Tabela II
RECURSOS DE SISTEMA OPERACIONAL EMPREGADOS NO PC3.

Recurso de SO	Implementação
Múltiplos processos	Visão (Python) + Núcleo (C++)
Threads	threadMonitorPipe, threadControleLED
Pipes (IPC)	FIFO nomeado em /tmp/gate_pipe
Sinais	SIGINT / SIGTERM (encerramento limpo)
Exclusão mútua	std::mutex (estado e hardware)
E/S eficiente	poll() com timeout (sem busy-wait)

3) *Pipeline de Visão e Prova de Vida*: O processo de visão opera em resolução reduzida (320×240) com pré-processamento por CLAHE [5] e detecção por Haar Cascades [4]. A identidade é estimada pelo classificador LBPH [7], e a prova de vida combina quatro métricas com pesos individuais: (i) variância do *Local Binary Pattern* (microtextura da pele); (ii) variância do Laplaciano (foco e microbordas); (iii) desvio-padrão da magnitude dos gradientes de Sobel (distribuição espacial); e (iv) razão de energia entre altas e baixas frequências via FFT (detecção de superfícies planas como telas e papel). Para evitar disparos falsos por oscilações de um único quadro, os vereditos são integrados por uma **janela deslizante** (deque de 10 quadros), exigindo consistência mínima de 6 em 10 quadros. O fluxograma completo do processamento de software é apresentado na Fig. 3.

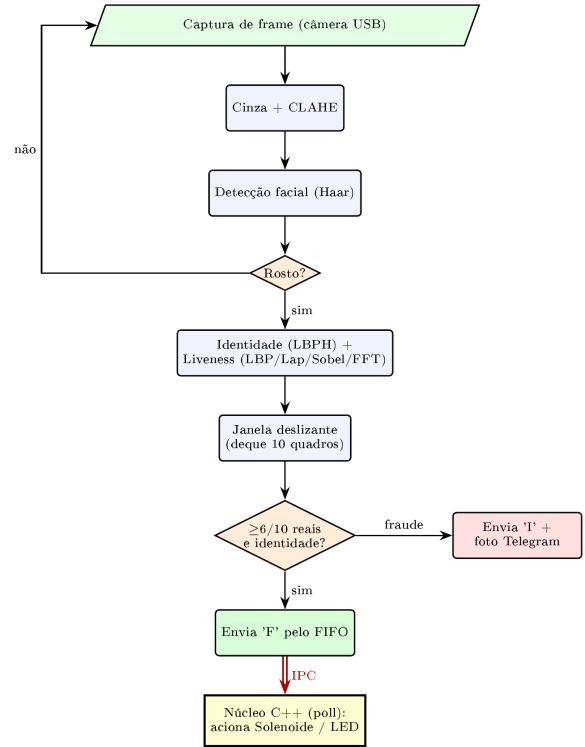


Figura 3. Fluxograma do fluxo de dados e processamento do software: da captura ao acionamento do hardware via IPC.

III. RESULTADOS EXPERIMENTAIS

A integração dos módulos de visão, controle e atuação foi validada experimentalmente sobre o protótipo montado. Esta seção apresenta tanto os comportamentos consolidados quanto os problemas diagnosticados durante a apresentação, que orientam os próximos ajustes.

A. Prova de Vida e Comunicação entre Processos

A *pipeline de liveness* integrada mostrou-se robusta e discriminante: para o usuário legítimo, o sistema retorna pontuação máxima $[4/4] \rightarrow [REAL]$ de forma estável, enquanto fotos e telas de celular são rejeitadas de forma consistente. A Listagem 4 ilustra um log típico de decisão por tendência temporal.

```
ID=1 (Conf:84.7) | LBP=9.8 | LAP=121.4 | FFT=0.83 [4/4] -> [REAL]
ID=1 (Conf:88.2) | LBP=9.1 | LAP=110.7 | FFT=0.79 [4/4] -> [REAL]
ID=1 (Conf:86.5) | LBP=9.4 | LAP=118.0 | FFT=0.81 [4/4] -> [REAL]
[ACESSO CONCEDIDO] Identidade e Liveness confirmados!
[C++] Sinal IPC: Autorizado! Abrindo Solenoide...
```

Listing 4. Log de execução: prova de vida e veredito por janela temporal.

A comunicação assíncrona entre processos (descriptor persistente com `poll()` em C++ e escrita `O_NONBLOCK` em Python) estabilizou completamente após a correção descrita na Seção II-B: não há mais perda de bytes nem janelas sem leitor ativo, e o consumo de CPU permaneceu baixo por não haver *busy-wait*.

B. Problemas Diagnosticados

1) *Falsos negativos de identidade*: Após a reconstrução do banco LBPH com uma rotina de treino multi-ângulo em 7 fases (210 amostras de 120×120 pixels, gerando um classificador de aproximadamente 20 MB), as distâncias de erro (*confidence*) retornadas pelo `predict()` para o usuário legítimo, sob a iluminação do laboratório, oscilaram entre **84,7 e 98,0**. Com o limiar de aceitação configurado de forma restrita (valores testados entre 70 e 90), o sistema passou a rejeitar o usuário correto, interpretando-o como intruso devido à sensibilidade do LBPH a variações finas de iluminação ambiente. Este é o principal ponto de calibração em aberto, tratado na Seção IV.

2) *Anomalia elétrica (LED fantasma)*: Logo após acomodar a fiação dentro da *case* impressa em 3D, o LED RGB passou a emitir uma luz fraca e mista (verde e vermelho simultâneos) e a solenoide deixou de atuar, mesmo com os logs de software registrando o envio e a recepção corretos dos comandos via `pinctrl`. O diagnóstico preliminar aponta uma falha de circuito físico — provavelmente o rompimento de um *jumper* ou um barramento de GND comum flutuante/desconectado —, coerente com o requisito de terra comum destacado na Seção II-A. Por ser uma falha de hardware (e não de lógica), a correção aplicada foi a simple troca dos jumpers.

IV. CONCLUSÕES

Este ponto de controle consolidou a transição de módulos isolados para um **protótipo integrado de tempo real**. O problema de uma portaria eletrônica autônoma e com privacidade foi atacado por uma arquitetura concorrente de dois processos — visão em Python e núcleo de controle em C++ — comunicando-se por *pipe* nomeado e empregando *threads*, *mutex*, sinais e a chamada `poll()` para sincronização eficiente. Os resultados confirmam que a prova de vida multi-métrica é robusta e que a camada de IPC ficou estável e econômica em CPU.

Entre os aprendizados, destacam-se duas lições centrais. Primeiro, que o classificador LBPH, por não ser invariante a iluminação, é o elo mais frágil da cadeia: a robustez do *anti-spoofing* não se traduz automaticamente em robustez de identidade. Segundo, que a confiabilidade de um sistema embarcado depende tanto do software quanto da integridade física das conexões — a anomalia do terra comum mostrou que um sistema logicamente correto pode falhar por uma conexão elétrica rompida.

Como **trabalhos futuros**, propõe-se:

- **Calibração do classificador**: introduzir uma classe negativa ($ID=2$) com rostos desconhecidos, forçando o LBPH a separar melhor as fronteiras de decisão, e normalizar a iluminação da ROI antes da extração de características para reduzir a sensibilidade às flutuações de luz do ambiente;
- **Robustez biométrica**: avaliar a migração do LBPH para *embeddings* faciais por rede neural, mais tolerantes a variações de pose e iluminação;

- **Senha temporária para entregadores**: implementar o código de uso único e validade limitada (caso iFood) sugerido no PC1, integrando-o ao mesmo canal de IPC;
- **Correção de hardware**: revisar a continuidade do barramento de terra comum e substituir conexões de *proto-board* por soldas definitivas na *case*.

REFERÊNCIAS

- [1] Ring LLC, “Ring video doorbell,” 2024. [Online]. Available: <https://ring.com/>
- [2] Google Store, “Google nest doorbell,” 2024. [Online]. Available: https://store.google.com/product/nest_doorbell
- [3] Itseez, “Open source computer vision library,” 2015. [Online]. Available: <https://opencv.org>
- [4] P. Viola and M. Jones, “Rapid object detection using a boosted cascade of simple features,” *CVPR*, 2001.
- [5] K. Zuiderveld, “Contrast limited adaptive histogram equalization,” *Graphics Gems IV*, 1994.
- [6] T. Ojala, M. Pietikainen, and T. Maenpää, “Multiresolution gray-scale and rotation invariant texture classification with local binary patterns,” *IEEE PAMI*, vol. 24, no. 7, pp. 971–987, 2002.
- [7] T. Ahonen, A. Hadid, and M. Pietikainen, “Face description with local binary patterns: Application to face recognition,” *IEEE PAMI*, vol. 28, no. 12, pp. 2037–2041, 2006.
- [8] W. R. Stevens and S. A. Rago, *Advanced Programming in the UNIX Environment*, 3rd ed. Addison-Wesley, 2013.

ANEXO I – ACESSO AO REPOSITÓRIO DO PROJETO

Devido à extensão e à natureza iterativa do desenvolvimento, o código-fonte integral (núcleo C++ e *pipeline* de visão em Python), o histórico de versões e os arquivos de configuração estão hospedados em ambiente Git público.

O repositório contém:

- **main.cpp**: núcleo de controle em C++ (*threads*, *mutex*, *poll*, sinais e acionamento via `pinctrl`);
- **vision.py**: *pipeline* de visão (detecção, *liveness* multi-métrica, LBPH e IPC);
- **treinar_e_ver.py**: treino multi-ângulo do classificador (7 fases, 210 amostras);
- **treinar_incremental.py**: ajuste incremental do modelo;
- **haarcascade_frontalface_default.xml**: modelo de detecção facial.

Observação: o classificador biométrico (`classificador_lbph.yml`, ≈ 20 MB) não é versionado junto ao texto por ser um artefato binário de grande porte e conter dados biométricos; ele é gerado localmente pela rotina de treino ou disponibilizado sob demanda no repositório.

Link de Acesso: <https://github.com/FuytearUnB/SOE-2026.1/tree/PC3>